Elasticity Efficiency in Cloud Storage using Auto Scaling Group

Project Owner: Swagata Sinha
B.Tech CSE Cloud Tech & Virtualization AWS (4th Semester)

Agenda

- Introduction to Cloud Elasticity and Auto Scaling
- Problem Statement and Motivation
- Related Work Overview
- Proposed System Architecture
- Auto Scaling Group Configuration and Methodology
- Elasticity Efficiency Analysis and Results
- Cost-Aware Evaluation
- Conclusion and Future Work

Problem Statement

Existing cloud auto-scaling approaches lack a practical, cost-aware evaluation of elasticity efficiency for storage-intensive workloads using Auto Scaling Groups, particularly under multi-metric and real-world cloud conditions.

- Cloud storage workloads are **highly dynamic and unpredictable**
- Traditional auto-scaling approaches often rely on **single-metric (CPU-only)** policies
- Existing studies focus mainly on **theoretical models or simulations**
- **Cost-aware evaluation** of elasticity is rarely performed in real cloud environments
- There is a lack of **practical, multi-metric analysis** of elasticity efficiency for **storage-intensive workloads** using **Auto Scaling Groups (ASGs)**

Pre-requisite and dependency

Prerequisites

- Basic understanding of **Cloud Computing concepts**
- Knowledge of **AWS services** (EC2, Auto Scaling Groups, CloudWatch)
- Familiarity with **virtual machines and storage (EBS)**
- Understanding of **performance metrics** (CPU, Network traffic)

Dependencies

- **AWS Cloud Platform** for deployment and testing
- **Auto Scaling Groups (ASG)** for elasticity management
- **CloudWatch** for monitoring and scaling triggers
- **Application Load Balancer (ALB)** for traffic distribution
- **AWS Cost Model / Pricing Calculator** for cost analysis

Objectives

- To study existing **cloud auto-scaling techniques** and identify their limitations for storage-intensive workloads
- To design and implement **multi-metric auto-scaling policies** using CPU utilization and network traffic
- To evaluate the effectiveness of **Auto Scaling Groups** in handling dynamic storage workloads
- To compare **static resource provisioning** with **ASG-based elastic provisioning** in terms of Performance and stability
- To propose and use a simple **Elasticity Efficiency Index (EEI)** for comparative evaluation
- To demonstrate **self-healing and availability** features of ASG-based cloud systems
- To identify gaps and propose a **storage-aware elasticity model** as future enhancement
- To analyze the **cost implications** of elasticity in cloud storage environments

Solutions:

- Multi-metric elastic scaling (CPU + Network)
- Storage-aware scaling model
- Multi-AZ resilient architecture
- Elasticity Efficiency Index (EEI)
- Self-healing cloud mechanism
- Static vs Dynamic performance comparison
- Cost-aware elasticity analysis

Solution 1:

Storage-Aware Scaling

- Extend ASG policies to include **EBS IOPS/throughput** and **EFS latency** metrics.
- Use **CloudWatch custom metrics** to monitor storage bottlenecks alongside CPU/network.
- Example: Scale out EC2 instances when **EBS volume queue length > threshold**, ensuring storage doesn't throttle compute elasticity.

Solution 2:

Multi-Metric Scaling Policies

Combine **CPU utilization, network input, and storage metrics** in scaling decisions.

Apply **target tracking scaling** for balanced elasticity (e.g., maintain 70% CPU + <10ms EFS latency).

Prevent over-scaling by using **step scaling** with gradual thresholds.

Solution 3:

Self-Healing Storage Integration

Extend health checks to include **EBS volume health** and **EFS mount status**.

Automate **instance replacement with attached volumes** using lifecycle hooks.

Use **Elastic Block Store snapshots** for quick recovery during instance replacement.

Solution 4:

Cost-Aware Elasticity

Implement **AWS Compute Optimizer** and **Cost Explorer** to analyze scaling efficiency.

Use **Spot Instances** for non-critical workloads to reduce cost while maintaining elasticity.

Apply **storage tiering** (S3 Standard → S3 IA → Glacier) for long-term elasticity in storage.

Solution 5:

Predictive Scaling

Leverage **AWS Auto Scaling predictive scaling** to forecast demand based on historical patterns.

Align storage provisioning (EBS/EFS) with predicted compute scaling events.

Example: Pre-warm EBS volumes before peak traffic to avoid latency spikes.

Solution 6:

Evaluation & Analysis Layer

Perform **Elasticity Efficiency Index (EEI)** calculations combining compute + storage metrics.

Run **cost-benefit analysis** for scale-out vs. scale-in events.

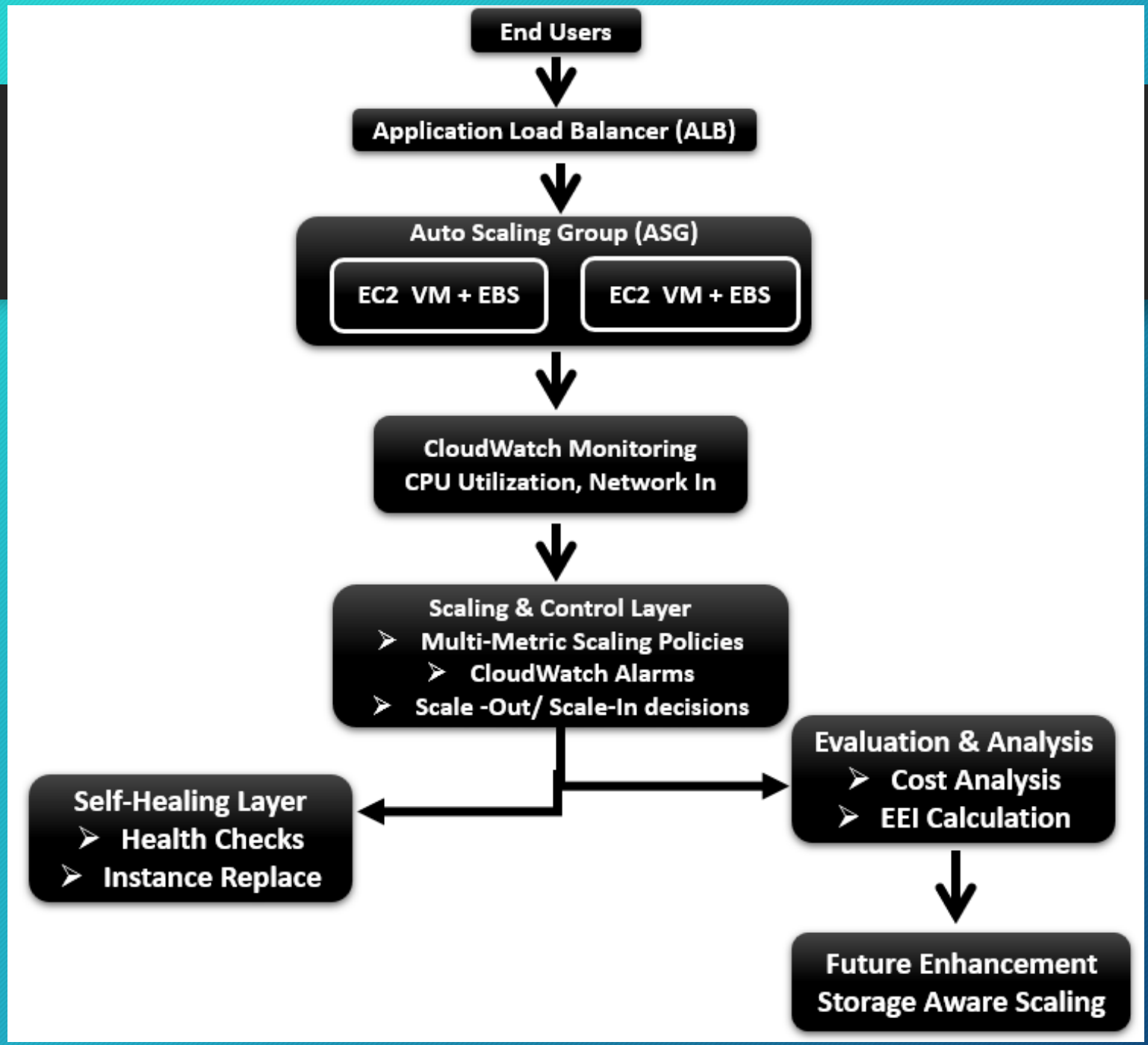
Use **AWS Trusted Advisor** for recommendations on underutilized or over-provisioned resources.

$$EEI = \frac{Average\ Utilization}{Cost}$$

System Architecture

- End users generate dynamic read/write requests for the cloud storage application.
- An Application Load Balancer (ALB) distributes incoming requests evenly across available resources to ensure load balancing and high availability.
- An Auto Scaling Group (ASG) manages multiple EC2 instances and dynamically adjusts the number of instances based on workload demand.
- Each EC2 instance is attached with Elastic Block Store (EBS) to support storage-intensive operations.
- AWS CloudWatch continuously monitors key metrics such as CPU utilization and network traffic, representing compute load and storage access behavior.
- A scaling and control layer uses CloudWatch alarms and multi-metric scaling policies to trigger scale-out and scale-in actions within the ASG.
- The architecture supports self-healing, where unhealthy or terminated instances are automatically replaced by the ASG to maintain service availability.
- An evaluation and analysis layer performs cost-aware elasticity analysis and computes the Elasticity Efficiency Index (EEI) to assess system performance.
- A storage-aware scaling mechanism is proposed as a future enhancement to further improve application-specific elasticity decisions.

Logical Flow



Outcomes

A. Static Environment

- **Fixed instance count**
- **High CPU spikes during load**
- **Resource wastage during idle**
- **No fault tolerance**
- **Constant cost**

B. ASG-Based Environment

- **Dynamic instance scaling**
- **Improved load distribution**
- **Reduced idle resource usage**
- **Automatic failure recovery**
- **Better cost-performance balance**

C. Quantitative Outcomes

- **Lower average CPU saturation**
- **Dynamic instance count graph**
- **Reduced idle-time cost**
- **Improved Elasticity Efficiency Index (EEI)**

Comparison of Existing Studies & Proposed Work

Aspect	Existing Studies	Limitations in Existing Work	Enhancement in This Work
Focus Area	General cloud auto-scaling and elasticity across diverse applications	Limited focus on storage-intensive cloud workloads	Focuses specifically on elasticity efficiency in cloud storage systems
Scaling Metrics	Primarily single-metric scaling (e.g., CPU utilization)	CPU-only metrics fail to capture storage access behavior	Implements multi-metric scaling using CPU utilization and network traffic to reflect storage workload characteristics
Scaling Approach	Mostly theoretical models or simulation-based evaluations	Lack of real-world implementation and experimental validation	Performs practical implementation using AWS Auto Scaling Groups (ASGs)
Adaptation Type	Reactive or conceptually hybrid scaling strategies	Delayed response and limited adaptability in real deployments	Uses event-driven ASG policies with CloudWatch alarms for responsive elasticity
Self-Adaptive Capability	Discussed conceptually (self-aware, self-adaptive systems)	Limited demonstration on commercial cloud platforms	Demonstrates self-healing behavior through ASG instance replacement
Cost Consideration	Cost-performance trade-offs discussed qualitatively	Lack of simple, reproducible cost-aware evaluation	Includes explicit cost-aware elasticity analysis using AWS cost estimates
Elasticity Evaluation	No standardized elasticity efficiency metrics	Difficult to quantitatively compare elasticity behavior	Proposes a lightweight Elasticity Efficiency Index (EEI) for comparative evaluation
Comparative Analysis	Often evaluates a single approach	Limited baseline comparison	Conducts static provisioning vs ASG-based provisioning comparative study
Application Awareness	Application-aware scaling identified as an open challenge	Storage-aware scaling largely unexplored	Proposes a storage-aware elasticity model as future enhancement
Practical Relevance	Strong theoretical contribution	Limited deployment-oriented insights	Bridges theory and practice through AWS-based experimental analysis

SDGs and Mapping

Mapping to SDG 12 (Responsible Consumption and Production)

This project supports SDG 12 by promoting responsible consumption of cloud resources through elastic auto-scaling, cost-aware analysis, and reduction of infrastructure waste.

Here's how this system aligns:

Aspect	How This Project Contributes
Efficient Resource Utilization	The project ensures that cloud resources are used only when needed by automatically scaling EC2 instances based on real workload demand, avoiding unnecessary usage.
Reduction of Over-Provisioning	Instead of keeping servers running all the time, the system scales down during low demand periods, which helps prevent wastage of cloud resources.
Cost-Aware Consumption	By analyzing cost along with performance, the project encourages smarter use of cloud services and helps users understand the cost impact of elasticity decisions.
Reduction of Energy and Operational Waste	Fewer idle servers mean less energy consumption in data centers, indirectly supporting environmentally responsible cloud operations.
Sustainable Cloud Operations	The use of auto-scaling and self-healing mechanisms supports long-term, efficient, and reliable cloud storage services without excessive resource consumption.

Risks

Risk	Description	Mitigation
Incorrect Scaling Thresholds	Poorly set CPU/network limits may cause over-scaling or under-scaling	Tune thresholds after observing initial workload
CloudWatch Delay	Monitoring latency may delay scaling action	Use shorter evaluation periods
Instance Launch Delay	New EC2 instances take time to boot	Keep minimum instance > 1
Over-Scaling	Too many instances increase cost	Set max capacity limit
Under-Scaling	Too few instances affect performance	Use multi-metric scaling

Scheduling

Component	Completion
AWS EC2	2 days
AWS EC2 + ASG + EBS	4 days
AWS CloudWatch	2 weeks (Monitoring)
Innovations	1 week
Research Paper	3 weeks


Thank
You